كيف تختبر بالأمثلة (سؤال مهم)

#	المؤشر	التحكم
1	تغير الحياة بالكلمات	ظاهر
2	تعطي مثال من تعدك	ظاهر
3	تقدر الحسب	ظاهر
4	أعلن السطلبة	ظاهر

Q1. What do "design" and "architecture" mean, and why is it described as "HOW" rather than "WHAT"?

Answer: Analysis produces the **WHAT** — the problem statement, the needs gathered from the user. Design produces the **HOW** — the solution structure that satisfies those requirements. The phrase from Grady Booch is: "design means the same thing as architecture, except that design connotes a smaller, more detailed piece of the problem world, while design invents structure that did not exist. In the two-part design process (slide 5), the customer drives the WHAT (conceptual design) and the system builders drive the HOW (technical design). So "design is HOW" means design is the act of choosing the structure to satisfy the requirements — it decides which structure, which modules, and which interfaces will be used, none of which are given by the requirements themselves.

التقرير: يعتبر Design هو الـ (WHAT vs HOW) التحليل هو الـ WHAT، وهذا أساس الفصل كله.

Q2. Explain the difference between *conceptual design* and *technical design*. Give one question each type answers.

Answer: The chapter splits design into two parts (Fig. 2, slide 5): **Conceptual design** belongs to the customer — it answers "What will the system look like to users?", "Where will the data come from?", "How will the system be offered?", "How will the reports and screens look?" (slide 6). It is the user's view of the solution. **Technical design** belongs to the system builders — it describes hardware configuration, software structure, communications interfaces, I/O, software architecture, and network architecture (slide 7). Both answer HOW (not WHAT), but at different levels; conceptual design is the external behaviour-level HOW, technical design is the internal construction-level HOW. Both must together translate requirements into a working system.

#	الوجه	Conceptual design	Technical design
1	تعن بعدم	العميل (Customer)	الباني (System builders)
2	السؤال	Where does data come from? How will screens look?	Hardware config, software architecture, I/O
3	المستوى	طرحي ما يراه المستخدم	داخلي، تقني، تجريدي

القسم A — الفهم التأسيسي · Modularity · Design (Abstraction)

Q1. What does "design is a problem-solving activity" mean, and why is it described as "HOW" rather than "WHAT"?

Answer: Analysis produces the **WHAT** — the problem statement, the needs gathered from the user. Design produces the **HOW** — the solution structure that satisfies those requirements. The phrase from Grady Booch is: "design means the same thing as architecture, except that design connotes a smaller, more detailed piece of the problem world, while design invents structure that did not exist. In the two-part design process (slide 5), the customer drives the WHAT (conceptual design) and the system builders drive the HOW (technical design). So "design is HOW" means design is the act of choosing the structure to satisfy the requirements — it decides which structure, which modules, and which interfaces will be used, none of which are given by the requirements themselves.

التقرير: يعتبر Design هو الـ (WHAT vs HOW) التحليل هو الـ WHAT، وهذا أساس الفصل كله.

Q2. Explain the difference between *conceptual design* and *technical design*. Give one question each type answers.

Answer: The chapter splits design into two parts (Fig. 2, slide 5): **Conceptual design** belongs to the customer — it answers "What will the system look like to users?", "Where will the data come from?", "How will the reports and screens look?" (slide 6). It is the user's view of the solution. **Technical design** belongs to the system builders — it describes hardware configuration, software structure, communications interfaces, I/O, software architecture, and network architecture (slide 7). Both answer HOW (not WHAT), but at different levels; conceptual design is the external behaviour-level HOW, technical design is the internal construction-level HOW. Both must together translate requirements into a working system.

#	الوجه	Conceptual design	Technical design
1	تعن بعدم	العميل (Customer)	الباني (System builders)
2	السؤال	Where does data come from? How will screens look?	Hardware config, software architecture, I/O
3	المستوى	طرحي ما يراه المستخدم	داخلي، تقني، تجريدي

القسم B — الفهم العميق: Coupling

Q5. What does coupling measure, and why is "no dependencies at all" not achievable in practice?

Answer: Coupling is the measure of the degree of interdependence between modules (slide 15). The chapter pictures three situations: uncoupled (no dependencies), loosely coupled (some dependencies) and highly coupled (many dependencies) (Fig. 5, slides 15–16). "No dependencies at all" is not achievable for any system that does real work, because useful systems require modules to cooperate — a module that computes nothing and talks to nobody is useless. Moreover, the chapter's own definition (slide 15) says coupling is zero only when one can be completely decoupled and whether coupling can be reduced until there is no coupling between modules. The realistic target is therefore minimum coupling, not zero coupling: the designer reduces dependencies down toward the data coupling and without eliminating the necessary cooperation between modules.

التعريف: تعريف Coupling (slides 15–16) — "درجة الترابط بين الوحدات" — كلما زاد الترابط، زاد الاعتماد وصعب التعديل.

Q10. Explain the six types of coupling in order from best to worst, with one sentence each.

Answer (from worst): The chapter fixes the order in Fig. 7 (slide 19).

#	النوع	المعنى بجزء جملة	الترتيب
1	Data coupling	وحدات تمرر بيانات بسيطة فقط عبر واجهة معلنة	Best (الأفضل)
2	Stamp coupling	تمرر بنية بيانات كاملة بدل الحقول الأقل مما تحتاج الوحدة المستقبلة (slide 17)	
3	Control coupling	الواحدات تمرر معلومات تتحكم بمنطق (علم) لضبط من وحدة وتتخاذل معلو التابع الثاني	
4	External coupling	ارتباط بتواصل بصيغة خارجية مشتركة على الكتلة (slide 17)	
5	Common coupling	وحدتان تشتركان في مطلقة بيانات عامة (Global) — تصبح بيانات عالمي لكل من تعدل أيضا (slide 17)	
6	Content coupling	وحدة تتغير بيانات وحدة أخرى أو تدخل التحكم إلى وحدة وحدة أخرى (slide 17)	Worst (الأسوأ)

التقرير: فرز أنواعه (slides 17–19) — الترتيب مثلثي "Best" و"Worst" تحت. الجدول يجمع النقط 87%.

Q11. Why is Data coupling the best and Content coupling the worst? Give the single principle that orders the whole scale.

Answer: Data coupling is the best because the two modules communicate by passing data only — and apart from that data channel they are independent (slide 20). Nothing hidden is shared, the dependency is one-directional, and the caller can see exactly what it is sending. Content coupling is the worst because the module reaches into another module's internals (slide 23): it changes the inside of module B, or control is passed from one module to the middle of another (slide 23), where module B branches into D even though D is supposed to be under C's control. The single ordering principle is **how much hidden, internal knowledge of the other module is required, and how bidirectional the dependency is**. Passing the minimum data requires the least knowledge and is most one-directional (best); reaching into internals requires the most knowledge and creates an invisible bidirectional link (worst). Every intermediate type sits between these two extremes on that same axis.

#	النوع	المعنى بجزء جملة	الترتيب
6	Logical cohesion	عناصر تجمع معلومة لأنها في نفس العملية المنطقية من النوع المتشابه (slide 30)	Worst (الأسوأ)
7	Coincidental cohesion	عناصر يجمعها فقط عملية في موضوعة (slide 30)	

التقرير: جدول Fig. 11 يقارن "Tight Best" و"Loose Worst" تحت. متوسطة Communicational موجود في فيديو (11 Fig.).

Q13. What is the difference between *procedural cohesion* and *sequential cohesion*?

Answer: Sequential cohesion exists when one element's *output* is the *input* of the next — the chapter's words: "Module A outputs some data which is the input to module B" (slide 32). The key is that the data is retained in the same procedure" (slide 29). Procedural cohesion exists when the instructions "although accomplish different tasks, yet have been combined because there is a specific order in which the tasks are to be executed" (slide 29). The difference is *data flow versus control flow*: sequential cohesion is welded together by actual data passing from step to step, while procedural cohesion is welded together only by the order of execution — the steps could run in that order but no data crosses between them. That is why sequential sits higher (position 2) than procedural (position 4): a real data dependency is a stronger reason for two elements to live in the same module than a merely conventional order.

التقرير: تعريف Sequential (slide 32) و Procedural (slide 29) — الفرق في بيانات (Sequential) أم ترتيب فقط فقط (Procedural).

Q19. Why does *logical cohesion* usually force control coupling on the caller?

Answer: Logical cohesion occurs in modules containing instructions "that appear to be related because they fall into the same logical class of function" (slide 30) — for example, a module containing all inputs and outputs. Because the module bundles several different actual tasks under one name, the caller must tell the module at runtime which task it wants, and in practice this is done by passing a control flag or selection value into the module. That selector is control coupling (slide 23). So logical cohesion at the callee *causes* control coupling at the level of the interface — the two defects come as a package. This is also exactly what the chapter's Exercise 5.4 asks: "If a module has logical cohesion, what is the likely effect on the way it has to interact with others?" The answer is control coupling, because the module cannot decide its own task.

التقرير: تعريف Logical (slide 30) و Control (slide 23) — النتيجة المباشرة بينهما سؤال التمارين مكتوبة.

Q20. Explain the relationship between cohesion and coupling as shown in Fig. 12, and why they are not independent.

Answer: Fig. 12 (slide 31) presents the two properties as two views of the same modular partition, with the textual rule "a software engineer must design the modules with goal of **high cohesion and low coupling**". They are not independent because they are produced by the same decision: where to draw the module boundaries. Draw a boundary through the middle of a functional task and you get two weakly cohesive modules that still need to communicate, so coupling rises. Draw the boundary around a whole functional task and its exclusive data, and you get one highly cohesive module with a small interface, so coupling falls. Hence the chapter's warning that if the software is not properly modularised, "a host of seemingly trivial enhancements or changes will result in the depths of the project" (slide 31) — improving cohesion correctly is the same act as reducing coupling.

القسم C — الفهم العميق: Cohesion

Q14. What does cohesion measure, and how is it different in direction from coupling?

Answer: Cohesion is "a measure of the degree to which the elements of a module are functionally related" (slide 25) — how strongly the parts *inside one module* belong together. Coupling measures the interdependence *between* modules (slide 15). The directional difference is therefore intra-module versus inter-module cohesion looks *inward* at one module's internal relationships, coupling looks *outward* between two modules. They are two views of the same partition of the system (Fig. 12, slide 31): splitting a module badly — low cohesion — produces pieces that need to exchange more data, which is exactly what raises coupling. Conversely, a badly placed module boundary forces unrelated elements into one and still leaves cohesion in both. The chapter's stated goal is therefore always "high cohesion and low coupling" (slide 31).

#	النوع	المعنى بجزء جملة	الترتيب
1	Functional cohesion	كل العناصر في وحدة واحدة تساهم في مهمة واحدة (slide 28)	Best (الأفضل)
2	Sequential cohesion	مخرجات عنصر تعتمد كمدخلات للعنصر التالي (slide 28)	
3	Communicational cohesion	عناصر الوحدة تعمل على نفس البيانات (مخرجة من عنصر تعتمد كـ)	
4	Procedural cohesion	عناصر مختلفة لكن متجمعة بسبب ترتيب تنفيذ محدد (slide 29)	
5	Temporal cohesion	عناصر مرتبطة بالوقت فقط (مثل عنصر الصدمة الزمنية) (slide 29)	

التقرير: تعريف Cohesion (slide 25) مقابل Coupling (slide 15) — 12 نقطة (slide 31) — سؤال جدول تحليل 87%.

Q17. Explain the seven types of cohesion from best to worst, with one sentence each.

Answer (best → worst): The chapter fixes the order in Fig. 11 (slide 27).

#	النوع	المعنى بجزء جملة	الترتيب
1	Functional cohesion	كل العناصر في وحدة واحدة تساهم في مهمة واحدة (slide 28)	Best (الأفضل)
2	Sequential cohesion	مخرجات عنصر تعتمد كمدخلات للعنصر التالي (slide 28)	
3	Communicational cohesion	عناصر الوحدة تعمل على نفس البيانات (مخرجة من عنصر تعتمد كـ)	
4	Procedural cohesion	عناصر مختلفة لكن متجمعة بسبب ترتيب تنفيذ محدد (slide 29)	
5	Temporal cohesion	عناصر مرتبطة بالوقت فقط (مثل عنصر الصدمة الزمنية) (slide 29)	

التقرير: تعريف Encapsulation (slide 39) — ومثال برمجي فعلي في Couplingوتحت تصنيع Abstract.

Q8. What is *information hiding* (encapsulation), and why is it usually listed as the same idea as "the module's interface is its contract"?

Answer: The chapter defines encapsulation as "the separation of the external aspects of an object from the internal implementation details of the object" (slide 69) and notes it is commonly referred to as **information hiding**. The reason it is the same idea as "the interface is its contract" is that once the external aspect is separated out, that external aspect is the only thing the rest of the system can legally depend on — in other words it becomes a contract. Internal implementation details are no longer visible, so nobody can couple to them, and content coupling becomes impossible by construction. This is why encapsulation is not just a nicety: it is the mechanism that makes the loose coupling (slide 15) and high cohesion (slide 31) the chapter is aiming for actually possible. Protecting the module's internals is the direct cause of reducing inter-module dependency.

التقرير: تعريف Abstraction (slide 71) — "التجريد يأخذ عنصر الأصالة المزدوجة" — amplification + elimination — وربطها بالمثال 23-25.

Q7. Explain the difference between *data abstraction* and *procedural abstraction*.

Answer: Both are abstraction mechanisms but they hide different things. **Procedural (functional) abstraction** hides the algorithm behind a named operation: the caller writes sort(records) and need not know which sort is used; the notation for this in the chapter is the structure chart's black box — "a black box means that functionality known to the user without knowledge of internal design" (slide 44) — and it is available at the statement level (slide 68). **Data abstraction** hides the representation of a data type behind a named type plus the set of operations allowed on it: the caller knows that a Student record can be created and can be done with it, but not how the fields are laid out or stored. An **abstract data type** is precisely a data abstraction whose permitted operations are defined without any one of them being privileged — the chapter's Exercise 5.17 asks the relationship between ADTs and classes, which is that a class is an ADT plus inheritance and the object model.

#	الوجه	Procedural abstraction	Data abstraction
1	ما يخفي	الخوارزمية/الطريقة	التمثيل الداخلي للبيانات
2	المثال في الفصل	Structure chart black box (ص 44)	Abstract Data Type / class
3	المثال	sort(records)	Student; محاسبة، معاملة فعلة

التقرير: فرق أساسي (slide 71) — والسؤال جدول مقارنة عند 41/48/61 — والسؤال الترتيب 87%.

Q9. What is *abstraction* and *generalisation*, and why do they work in opposite directions?

Answer: Abstraction and generalisation are both ways of managing complexity, but they move in opposite directions. **Abstraction** is the elimination of the irrelevant and the amplification of the essentials" (slide 69) — complexity is managed by deliberately discarding the details that do not matter for the current purpose, and by emphasising the few features that do. The phrase captures both halves of the act: it is not merely deletion (elimination) but also highlighting (amplification). In OOD this is applied at several scales — the class abstracts the common structure and behaviour of many objects; the interface abstracts an operation's signature while leaving its implementation to subclasses (polymorphism, slide 58); and inheritance pulls the important parts of several classes up into a superclass (slide 79, slide 80). In every case the point is the reader must be able to think about the design without carrying irrelevant detail in working memory.

التقرير: تعريف Abstraction (slide 71) — والسؤال يشرح عنصر الأصالة المزدوجة "amplification + elimination" وربطها بالمثال 23-25.

Q7. Explain the difference between *data abstraction* and *procedural abstraction*.

Answer: Both are abstraction mechanisms but they hide different things. **Procedural (functional) abstraction** hides the algorithm behind a named operation: the caller writes sort(records) and need not know which sort is used; the notation for this in the chapter's structure chart's black box — "a black box means that functionality known to the user without knowledge of internal design" (slide 44) — and it is available at the statement level (slide 68). **Data abstraction** hides the representation of a data type behind a named type plus the set of operations allowed on it: the caller knows that a Student record can be created and can be done with it, but not how the fields are laid out or stored. An **abstract data type** is precisely a data abstraction whose permitted operations are defined without any one of them being privileged — the chapter's Exercise 5.17 asks the relationship between ADTs and classes, which is that a class is an ADT plus inheritance and the object model.

#	الوجه	Procedural abstraction	Data abstraction
1	ما يخفي	الخوارزمية/الطريقة	التمثيل الداخلي للبيانات
2	المثال في الفصل	Structure chart black box (ص 44)	Abstract Data Type / class
3	المثال	sort(records)	Student; محاسبة، معاملة فعلة

التقرير: تصنيف من خلص التصميم — سلالة 5/6/7 وسؤال جدول مقارنة لتفادات تشابك 87%.

Q3. Explain the four objectives of a software design, and why "at the right level" is not the same as "correct".

Answer: The chapter lists four objectives (slide 8): the design must be **Correct & complete** (it satisfies every requirement and leaves nothing out), **Understandable** (a reader can grasp the design without reverse-engineering it), **At the right level** (detailed is appropriate — neither too abstract to implement, nor so detailed it becomes code), and **Maintainable** (it can be changed later with contained cost). "At the right level" is not the same as "correct" because correctness is about content — does the design satisfy the requirements — while level is about packaging: a design can be perfectly correct yet still be written at the wrong level of detail. For instance, describing an entire subsystem in one line, or writing actual source statements inside the design document, both result in "incorrect" in context. It would fail the reader, because neither can be implemented or reviewed properly. Level is a requirement of design, not a property of correctness.

التقرير: التحديد بين خلص التصميم — سلالة 5/6/7 وسؤال جدول مقارنة لتفادات تشابك 87%.

Q4. What is modularity, and why is a module's interface more important than its implementation?

Answer: A modular system consists of well-defined, manageable units with well-defined interfaces among them (slide 11), and the chapter expands it as "the ease with which a system can be built" and "simpler from outside than from the inside" (slide 12). The interface is more important than the implementation because the interface is the only thing the rest of the system depends on. If only the interface is depended upon, the implementation may be changed freely without rippling outward — which is exactly what encapsulation/information hiding achieves (slide 69). The chapter's own coupling scale confirms this: the best coupling (data) depends only on the parameters crossing the interface, while the worst (content) reaches inside an implementation. So a module's value is measured by how cleanly its interface stands alone.

التقرير: تعريف Modularity — خصائص الواجهة — نقطة الانتشار الأعلى: تغير تصميم جدول Fig. 4.

Q5. Explain the modularity vs software cost relationship (the U-curve) and what it warns you against.

Answer: The modularity-versus-cost figure (Fig. 4, slide 14) plots "Cost of Effort" against "Number of Modules" and produces a **U-shaped curve**: cost is high when there are too few modules (each module is huge and hard to understand and change), falls to a minimum at an intermediate number of modules (the "Region of Minimum Cost"), then rises again as the number of modules grows further because the **cost-to-integrate** modules begins to dominate — more modules mean more communication, testing and coordination work. The chart's warning is that "more modules" is not automatically better: modularity has an optimum, and both extremes — under-modularising and over-modularising — are expensive. To see the bottom of the U, not at either arm.

التقرير: تعريف Region of Minimum Cost — منحنى لما وصفين Fig. 4 (سلالة 14) — يجمع النقط الخطأ "كل ما زادت الوحدات = أفضل".

Q6. What does "abstraction" mean in object-oriented design, and why does the chapter call it "the elimination of the irrelevant and the amplification of the essentials"?

التقرير: تعريف Abstraction (slide 71) — "التجريد يأخذ عنصر الأصالة المزدوجة" — amplification + elimination — وربطها بالمثال 23-25.

Q12. What is the difference between *stamp coupling* and *data coupling*, and why does passing a whole record hurt?

Answer: Data coupling passes only the *data actually needed* — typically simple values, one datum per parameter. Stamp coupling passes a complete *data structure* — an entire record or object — from one module to another (slide 20). Passing the whole record hurts because the receiving module now has access to far more than it requires: it can read, and its maintainers designers even depend on, fields that were never necessary. That extra exposure means (a) the receiving module's behaviour can silently begin to depend on unrelated fields, and (b) if the record's layout changes, the receiving module may need to change even though the datum it actually used did not. For example, if a Student record has fields for name, address, and GPA, but a module only needs the student ID rather than the whole student record. Stamp coupling is still better than control, external, common or content coupling, but it is worse than passing exactly what is needed.

التقرير: سلالة 20 + 21 وسبا التركيب. ورد أيضا في Fig.6 (سلالة 18) — تمرين Exercise 5.2.

Q13. Explain *control coupling* and why a flag makes two modules depend on each other's logic.

Answer: Control coupling occurs when modules communicate by passing *control information* — "usually accomplished by means of flags that are set by one module and reacted upon by the dependent module" (slide 21). A flag makes the two modules depend on each other's logic because the called module must contain a *branch* that knows what to do for each flag value, and the calling module must know which flag values are required. The chapter's own example is a sort routine with a flag telling it whether to sort in ascending or descending order (slide 21). A flag makes the two modules depend on each other's logic because the called module must contain a branch that knows what to do for each flag value, and the calling module must know which flag values are required. The chapter's own example is a sort routine with a flag telling it whether to sort in ascending or descending order (slide 21). A flag makes the two modules depend on each other's logic because the called module must contain a branch that knows what to do for each flag value, and the calling module must know which flag values are required. The chapter's own example is a sort routine with a flag telling it whether to sort in ascending or descending order (slide 21).

التقرير: تعريف Control coupling (slide 21) — سؤال تمرين Exercise 5.4 (سلالة 17).

Q14. Explain *common coupling* using the "global data area" example, and why the chapter says a change forces "tracing back to all modules".

Answer: With common coupling, modules A and B share named data — "global data areas are commonly found in programming languages", and "making a change to the common data means tracing back to all the modules which access that data to evaluate the effect of changes" (slide 21, with an example in Fig. 8, slide 22). A change forces tracing back to all accessing modules because the global area has no single owner and no visible interface nobody can tell, simply by reading a module, which other modules depend on each value they touch. Any modification therefore requires an exhaustive search of every module that reads or writes the shared area. This is why common coupling ranks 5th — worse than content coupling but better than complete: the dependency is real and bidirectional, but at least it is a *named shared region* rather than another module's private internals.

التقرير: تعريف Common coupling (slide 21) — مثال Fig. 8 — الصفحة الخاصة بك مثال واحد — تحقق أن الموصولات تقدر أن تظهر.

Q15. Explain *content coupling* with the chapter's own example, and why it is called "invisible change".

التقرير: تعريف Content coupling (slide 23) — Fig. 9 و Fig. 10 — تحقق بالواجهة. ذكرت التعليم بكيف صادفة Fig. 10 مجاورة.

Q16. What does cohesion measure, and how is it different in direction from coupling?

Answer: Cohesion is "a measure of the degree to which the elements of a module are functionally related" (slide 25) — how strongly the parts *inside one module* belong together. Coupling measures the interdependence *between* modules (slide 15). The directional difference is therefore intra-module versus inter-module cohesion looks *inward* at one module's internal relationships, coupling looks *outward* between two modules. They are two views of the same partition of the system (Fig. 12, slide 31): splitting a module badly — low cohesion — produces pieces that need to exchange more data, which is exactly what raises coupling. Conversely, a badly placed module boundary forces unrelated elements into one and still leaves cohesion in both. The chapter's stated goal is therefore always "high cohesion and low coupling" (slide 31).

#	النوع	المعنى بجزء جملة	الترتيب
1	Functional cohesion	كل العناصر في وحدة واحدة تساهم في مهمة واحدة (slide 28)	Best (الأفضل)
2	Sequential cohesion	مخرجات عنصر تعتمد كمدخلات للعنصر التالي (slide 28)	
3	Communicational cohesion	عناصر الوحدة تعمل على نفس البيانات (مخرجة من عنصر تعتمد كـ)	
4	Procedural cohesion	عناصر مختلفة لكن متجمعة بسبب ترتيب تنفيذ محدد (slide 29)	
5	Temporal cohesion	عناصر مرتبطة بالوقت فقط (مثل عنصر الصدمة الزمنية) (slide 29)	

التقرير: تعريف Cohesion (slide 25) مقابل Coupling (slide 15) — 12 نقطة (slide 31) — سؤال جدول تحليل 87%.

Q17. Explain the seven types of cohesion from best to worst, with one sentence each.

Answer (best → worst): The chapter fixes the order in Fig. 11 (slide 27).

#	النوع	المعنى بجزء جملة	الترتيب
1	Functional cohesion	كل العناصر في وحدة واحدة تساهم في مهمة واحدة (slide 28)	Best (الأفضل)
2	Sequential cohesion	مخرجات عنصر تعتمد كمدخلات للعنصر التالي (slide 28)	
3	Communicational cohesion	عناصر الوحدة تعمل على نفس البيانات (مخرجة من عنصر تعتمد كـ)	
4	Procedural cohesion	عناصر مختلفة لكن متجمعة بسبب ترتيب تنفيذ محدد (slide 29)	
5	Temporal cohesion	عناصر مرتبطة بالوقت فقط (مثل عنصر الصدمة الزمنية) (slide 29)	

التقرير: سلالة 14 + تمرين 5.13 — فرق "بنية الثقة" مقابل "تسلسل تنفيذ".

Q23. Explain what the SDD is and why the chapter calls it "the blueprint for the implementation activity".

Answer: An SDD (Software Design Description) is "a representation of a software system that is used as a medium for communicating software design information" (slide 50) and is created "to facilitate analysis, planning, implementation and decision making" (slide 51). The chapter states that "the SDD shows how the software system will be structured to satisfy the requirements identified in the SRS... it is basically the translation of requirements into a description of the software structure, software components, interfaces, and data necessary for the implementation phase. Hence, SDD becomes the blue print for the implementation activity" (slide 52). Calling it a blueprint means it is the authoritative reference that implementers build from and that correctness is judged against — it is not a summary produced after the fact but the source specification for coding.

التقرير: تعريف SDD (slides 50–52) — الاستعارة "Blueprint" مذكورة حرفيا مطلوبة.

Q24. Explain the four design views of the SDD, and why more than one view is needed at all.

Answer: Table 2 (slide 57) organizes the SDD into four views. **Decomposition description** — "partitions the system into design entities (e.g. hierarchical decomposition diagram, natural language). **Dependency description** — describes relationships among design entities (structure chart, data flow diagram, sequence diagram). **Interface description** — everything a designer, developer or tester needs in order to use each entity (interface files, parameter tables). **Detail description** — the internal design details of an entity (flow charts, PDL). More than one view is necessary because, as the chapter says, "each design description may have a number of entities that are considered the essential aspects of a specific software system, and different project activities need different subsets of these entities. Therefore, an implementer needs details, an architect needs the decomposition. A single flat document would force every reader to wade through irrelevant information.

#	الوجه	Scope	Design View
1	Decomposition description	تقسيم النظام لمكونات	Hierarchical decomposition diagram
2	Dependency description	العلاقات الموجهة بمكونها	Structure chart, DFDs
3	Interface description	كل ما يحتاجه المستخدم الواجهة	Interface files, parameter tables
4	Detail description	التفاصيل الداخلية للوحدة	Flow charts, PDL

التقرير: Table 2 (سلالة 57) — أربع وجهات. قدمت مقترحة سلالة 54 (slide 52–53) — الاستعارة "Blueprint" مذكورة حرفيا مطلوبة.

Q25. Explain what the ten design entity attributes are, and why an entity needs all of them rather than just a name.

Answer: The chapter lists ten attributes of a design entity (Slides 53): Identification, Type, Purpose, Function, Subordinates, Dependencies, Interface, Resources, Processing, and Data. A name alone is insufficient because a design entity is "an element of a design that is structurally and functionally distinct from other elements and that is separately named and referenced" (slide 51) — being named is only one of its properties, not its whole identity. Each attribute answers a different question a downstream reader will ask. *Identification* — which entity is this; *type* — what kind of entity; *Purpose* — why it exists; *Function* — what it does; *Subordinates* — what it decomposes into; *Dependencies* — what it relies on; *Interface* — how to use it; *Resources* — what it consumes;

#	العلاقة	المعنى	الأعلاه
3	Aggregation	علاقة "كل بجزء" (whole-part)	أقوى من Association
4	Generalization	علاقة بين صنفين	هي الأضعف إلى الأقوى

التقرير: الجدول Fig.77/78 — أنواع "أقوى" — القوة Dependency. Association هي Aggregation هي (whole-part) Generalization.

Q21. Explain what a *use case model* captures, and why it is the first step of the OOD process.

Answer: The first step in the analysis and design of an object-oriented system (Fig. 23, slide 72) is to "create use case models". A use case model captures "the behaviour of the system as seen by its users. We should then write the use case and draw the use case diagrams" (slide 73). A use case model therefore captures who interacts with the system (the actors) and what pools they possess (the use cases), plus the relationships that separates them. It comes first because everything downstream depends on the boundary it fixes: the interaction diagrams "show how the use cases are realized" (slide 74), the class diagram needs the entities those use cases manipulate, and the state chart needs the objects whose lifecycles matter. In other words, establishing actors and system scope first prevents the model from growing unbounded, which is the model-level equivalent of avoiding coincidental cohesion.

التقرير: سؤال Fig.72/73 — وسبا Step 1 من طريقة OOD. وتحدد Actors وتحدد. التركيز قبل أن تقدر أي آخر.

Q22. Explain the difference between a *use case diagram* and a *sequence diagram*.

Answer: A use case diagram, produced in step 1 of the OOD process (slide 73), shows the **static scope of interaction** — the actors outside the system and the goals/use cases the system provides for them. It says nothing about ordering or internal messages. A sequence diagram is one of the **interaction diagrams** produced in step 3 (slide 75), and an "interaction diagram shows an interaction, consisting of a set of objects and their relationships, including the messages that may be dispatched among them. Interaction diagrams address the dynamic view of a system" (slide 75). So the difference is scope-and-structure versus time-and-messages: the use case diagram answers "what can be done and by whom", while the sequence diagram answers "in what order do those specific objects exchange which messages to accomplish one use case". The chapter explicitly says sequence and collaboration diagrams are drawn *for every use case*.

#	الوجه	Use case diagram	Sequence diagram
1	السياقة	Step 1 من طريقة OOD	Step 3 من طريقة OOD
2	يعرض	actors + use cases	messages + Objects ترتيب زمني
3	النوع	Static scope	Dynamic view (زمني)
4	السؤال	ماذا يمكن أن يفعل وأين؟	بأي ترتيب تتبادل الكائنات الرسائل؟

التقرير: سلالة 73 مقابل 74 — الأول سلالة الثاني عرض تبادلهما زمنية وسائل.

Q33. Why does the chapter say OOD is "orthogonal" to function-oriented design, and what does each begin with?

التقرير: تعريف Orthogonal (slide 76) — "مستقل عن بعضهما" — تعبير عن Fig.24 — تعريف.

Q26. What is an *object* in OOD, and how is it different from a *class*?

Answer: An object is "an entity able to save a state (information) and which offers a number of operations (behavior) to either examine or affect this state. An object is characterized by number of operations and a state which remembers the effect of these operations" (slide 60). A class is "a set of objects that share a common structure and a common behavior" (slide 62) — "classes are useful because they act as a blueprint for objects" (slide 62). The difference is instance versus description: the class is the template that specifies which attributes and operations each object will have, and each object is one concrete instance of that template. Each attribute value is the chapter's example being Indira, Sarmi, Marak and Indigo as instances of the class "cat" (Fig. 20, slide 63). All objects of a class share the same operations, but each holds its own state.

#	الوجه	Object	Class
1	الطبيعة	نسخة/instance	تعريف/blueprint
2	العلاقة	له عدد محدد من الحالة	يحدد السلوك والحالة المشتركة
3	المثال في الفصل	Indira, Sarmi	Class "cat"
4	المصدر	سلالة 59	سلالة 62

التقرير: سلالة 59 + 62 + 63 — تعريف Object و Class — مثال Indira, Sarmi, Marak, Indigo — Blueprint instance — السؤال Fig.20 (سلالة 63).

Q27. What is an *abstract data type*, and what is its relationship to a class?

Answer: An abstract data type (ADT) is a data structure whose permitted operations are defined while its internal representation is hidden — the user of the type knows what can be done with it, not how it is stored. A class is an API enriched with the object model: the chapter defines a class as a set of objects sharing a common structure and behavior (slide 62), and it says that "a class is an ADT plus inheritance" (slide 80). So ADT and operations as "a function or transformation that may be applied to or by objects in a class" (slide 60). The relationship is therefore that a class is an ADT plus the additional mechanisms of inheritance, polymorphism and the notion of an instance — which is exactly what the chapter's Exercise 5.17 asks: "What is the relationship between abstract data types and classes?" Classes implement the ADT idea concretely.

التقرير: /inheritance/polymorphism/references = ADT + Class — 5.17 تمرين — سلالة 62/65 — تعريف Class.

Q28. Explain *inheritance* using the chapter's Shape example, and state precisely what is inherited.

Answer: Inheritance is described as the mechanism by which "low level classes (known as subclasses or derived classes) inherit state and behavior from this high level class (known as a super class or base class)" (slide 68). In the chapter's example, the subclasses Circle, Square and Triangle "all inherit the ability to move, and, by abstraction, we might want to think of a picture as made up of shapes... we do not care that one shape is a

square and the other is a triangle as long as both can draw themselves" (slide 67). The important parts are therefore pulled out into a new class called **Shape** (Fig. 25, slide 68), and the subclasses inherit from it. Precisely, what is inherited is **state (attributes) and behaviour (operations)** — not implementation choices made inside the subclasses. This lets the design express sub-types generically, which the chapter says is the key mechanism to a completely-reducing mechanism (slide 69).

التقرير: تعريف Inheritance (slide 68) — مثال Shape — الوراثة في الشكل — تعريف superclass و subclass — الوجوه — 23–67 — 23–68 — 67/68.

Q29. Explain the difference between *inheritance* and *polymorphism*, and why one can exist without the other.

Answer: Inheritance is a structural relationship — subclasses derive state and behaviour from a superclass (slide 68). Polymorphism is a behavioural mechanism — "when we abstract just the interface of an operation and leave the implementation to subclasses it is called a polymorphic operation and process is called polymorphism" (slide 69). Polymorphism is also "the ability of a variable to contain more than one type of object" (slide 70). The answer is yes, a subclass can inherit behaviour without polymorphism (Exercise 5.18): the answer is yes, a subclass can inherit behaviour without polymorphism (Exercise 5.18): The answer is yes, a subclass can inherit behaviour without polymorphism (Exercise 5.18): The answer is yes, a subclass can inherit behaviour without polymorphism (Ex